hazardandRFtime1Run

April 11, 2026

1 LendingClub discrete-time hazard model project

2 File: data/raw/accepted_2007_to_2018Q4.csv.gz

3

4 Workflow:

5 1. Load and clean data

6 2. Define event and censoring

7 3. Create duration

8 4. Expand to loan-month panel

9 5. Train/valid/test split

10 6. Logistic hazard model

11 7. Random forest comparison

12 8. Cumulative PD

13 9. Expected loss / reserve / pricing summaries

[1]:
```
import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from pathlib import Path

from sklearn.compose import ColumnTransformer
```

```python
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, average_precision_score

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)
```

[2]:
```python
DATA_DIR = Path("data")
RAW_DIR = DATA_DIR / "raw"
PROCESSED_DIR = DATA_DIR / "processed"
PROCESSED_DIR.mkdir(parents=True, exist_ok=True)

accepted_path = RAW_DIR / "lendingclub" /"accepted_2007_to_2018Q4.csv.gz"

# Runtime controls
USE_SAMPLE = True          # set to False if you want the full dataset
SAMPLE_SIZE = 50000        # increase later if your machine can handle it
RANDOM_STATE = 42
```

[3]:
```python
df_raw = pd.read_csv(
    accepted_path,
    low_memory=False,
    compression="gzip"
)

print("Raw shape:", df_raw.shape)
print("First 20 columns:", df_raw.columns.tolist()[:20])
```

```
Raw shape: (2260701, 151)
First 20 columns: ['id', 'member_id', 'loan_amnt', 'funded_amnt',
'funded_amnt_inv', 'term', 'int_rate', 'installment', 'grade', 'sub_grade',
'emp_title', 'emp_length', 'home_ownership', 'annual_inc',
'verification_status', 'issue_d', 'loan_status', 'pymnt_plan', 'url', 'desc']
```

[4]:
```python
key_columns = [
    "id",
    "issue_d",
    "loan_status",
    "term",
    "loan_amnt",
    "installment",
    "emp_length",
    "home_ownership",
    "annual_inc",
    "verification_status",
```

```
    "purpose",
    "addr_state",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_range_low",
    "fico_range_high",
    "earliest_cr_line",
    "last_pymnt_d",
    "last_credit_pull_d",
]

missing_cols = [c for c in key_columns if c not in df_raw.columns]
print("Missing columns:", missing_cols)
```

```
Missing columns: []
```

[5]:
```
cols = [c for c in key_columns if c in df_raw.columns]
df = df_raw[cols].copy()

print("Working shape:", df.shape)
df.head()
```

```
Working shape: (2260701, 27)
```

[5]:
```
         id   issue_d loan_status        term  loan_amnt  installment  \
0  68407277  Dec-2015  Fully Paid   36 months     3600.0       123.03
1  68355089  Dec-2015  Fully Paid   36 months    24700.0       820.28
2  68341763  Dec-2015  Fully Paid   60 months    20000.0       432.66
3  66310712  Dec-2015     Current   60 months    35000.0       829.90
4  68476807  Dec-2015  Fully Paid   60 months    10400.0       289.91

  emp_length home_ownership  annual_inc verification_status  \
0  10+ years       MORTGAGE     55000.0        Not Verified
1  10+ years       MORTGAGE     65000.0        Not Verified
2  10+ years       MORTGAGE     63000.0        Not Verified
3  10+ years       MORTGAGE    110000.0     Source Verified
4    3 years       MORTGAGE    104433.0     Source Verified

              purpose addr_state    dti  delinq_2yrs  inq_last_6mths  \
0  debt_consolidation         PA   5.91          0.0             1.0
```

```
1      small_business         SD  16.06          1.0             4.0
2    home_improvement         IL  10.78          0.0             0.0
3  debt_consolidation         NJ  17.06          0.0             0.0
4      major_purchase         PA  25.37          1.0             3.0

   open_acc  pub_rec  revol_bal  revol_util  total_acc  mort_acc  \
0       7.0      0.0     2765.0        29.7       13.0       1.0
1      22.0      0.0    21470.0        19.2       38.0       4.0
2       6.0      0.0     7869.0        56.2       18.0       5.0
3      13.0      0.0     7802.0        11.6       17.0       1.0
4      12.0      0.0    21929.0        64.5       35.0       6.0

   pub_rec_bankruptcies  fico_range_low  fico_range_high earliest_cr_line  \
0                   0.0           675.0            679.0         Aug-2003
1                   0.0           715.0            719.0         Dec-1999
2                   0.0           695.0            699.0         Aug-2000
3                   0.0           785.0            789.0         Sep-2008
4                   0.0           695.0            699.0         Jun-1998

  last_pymnt_d last_credit_pull_d
0     Jan-2019           Mar-2019
1     Jun-2016           Mar-2019
2     Jun-2017           Mar-2019
3     Feb-2019           Mar-2019
4     Jul-2016           Mar-2018
```

[6]:
```
if USE_SAMPLE:
    df = df.sample(SAMPLE_SIZE, random_state=RANDOM_STATE).copy()

print("Shape after optional sampling:", df.shape)
```

```
Shape after optional sampling: (50000, 27)
```

[7]:
```
date_cols = ["issue_d", "earliest_cr_line", "last_pymnt_d",␣
 ↪"last_credit_pull_d"]

for c in date_cols:
    if c in df.columns:
        df[c] = pd.to_datetime(df[c], format="%b-%Y", errors="coerce")

df["fico_avg"] = (df["fico_range_low"] + df["fico_range_high"]) / 2

df["term_num"] = (
    df["term"]
    .astype(str)
    .str.extract(r"(\d+)")
    .astype(float)
```

```
)

df["credit_history_years"] = (
    (df["issue_d"] - df["earliest_cr_line"]).dt.days / 365.25
)

df[["issue_d", "loan_status", "term", "term_num", "fico_avg",␣
 ↪"credit_history_years"]].head()
```

[7]:

```
           issue_d loan_status        term  term_num  fico_avg  \
392949  2015-02-01     Current   60 months      60.0     737.0
1273506 2014-05-01  Fully Paid   36 months      36.0     687.0
324024  2015-04-01  Fully Paid   36 months      36.0     782.0
2066630 2017-12-01  Fully Paid   36 months      36.0     742.0
477199  2018-02-01     Current   60 months      60.0     717.0

         credit_history_years
392949              33.336071
1273506             13.081451
324024              11.581109
2066630             12.501027
477199              18.590007
```

[8]:

```
status_counts = df["loan_status"].value_counts(dropna=False)
status_counts
```

[8]:

```
loan_status
Fully Paid                                          23771
Current                                             19370
Charged Off                                          6015
Late (31-120 days)                                    505
In Grace Period                                       186
Late (16-30 days)                                     108
Does not meet the credit policy. Status:Fully Paid     24
Does not meet the credit policy. Status:Charged Off    19
NaN                                                     2
Name: count, dtype: int64
```

[9]:

```
# Basic event definition:
# event = 1 for default-like outcomes
# event = 0 otherwise

default_statuses = {
    "Charged Off",
    "Default",
}
```

```
df["event"] = df["loan_status"].isin(default_statuses).astype(int)

# Use last payment date if available; otherwise use last credit pull date
df["end_date"] = df["last_pymnt_d"]
mask_missing_end = df["end_date"].isna()
df.loc[mask_missing_end, "end_date"] = df.loc[mask_missing_end,␣
↪"last_credit_pull_d"]

df[["loan_status", "event", "issue_d", "last_pymnt_d", "last_credit_pull_d",␣
↪"end_date"]].head()
```

[9]:

```
        loan_status  event    issue_d last_pymnt_d last_credit_pull_d  \
392949      Current      0 2015-02-01   2019-03-01         2019-03-01
1273506  Fully Paid      0 2014-05-01   2017-06-01         2017-08-01
324024   Fully Paid      0 2015-04-01   2017-03-01         2019-02-01
2066630  Fully Paid      0 2017-12-01   2018-10-01         2018-10-01
477199      Current      0 2018-02-01   2019-03-01         2019-03-01

          end_date
392949  2019-03-01
1273506 2017-06-01
324024  2017-03-01
2066630 2018-10-01
477199  2019-03-01
```

[10]:

```
def month_diff(start, end):
    return (end.dt.year - start.dt.year) * 12 + (end.dt.month - start.dt.month)

df = df[df["issue_d"].notna()].copy()
df = df[df["end_date"].notna()].copy()

df["duration_months"] = month_diff(df["issue_d"], df["end_date"])
df["duration_months"] = df["duration_months"].clip(lower=1)

print(df.shape)
df[["id", "issue_d", "end_date", "duration_months", "event"]].head()
```

```
(49998, 33)
```

[10]:

```
                id    issue_d   end_date  duration_months  event
392949    39651438 2015-02-01 2019-03-01               49      0
1273506   16411620 2014-05-01 2017-06-01               37      0
324024    45122316 2015-04-01 2017-03-01               23      0
2066630  125356772 2017-12-01 2018-10-01               10      0
477199   128490686 2018-02-01 2019-03-01               13      0
```

[11]:
```
df = df[df["term_num"].isin([36, 60])].copy()
df = df[df["duration_months"] >= 1].copy()
df = df[df["fico_avg"].notna()].copy()

print("After cleaning/filtering:", df.shape)
```

```
After cleaning/filtering: (49998, 33)
```

[12]:
```
print("Event rate:")
print(df["event"].value_counts(normalize=True))

print("\nLoan status counts:")
print(df["loan_status"].value_counts(dropna=False).head(20))

print("\nDuration summary:")
print(df["duration_months"].describe())
```

```
Event rate:
event
0    0.879695
1    0.120305
Name: proportion, dtype: float64

Loan status counts:
loan_status
Fully Paid                                            23771
Current                                               19370
Charged Off                                            6015
Late (31-120 days)                                      505
In Grace Period                                         186
Late (16-30 days)                                       108
Does not meet the credit policy. Status:Fully Paid       24
Does not meet the credit policy. Status:Charged Off      19
Name: count, dtype: int64

Duration summary:
count    49998.000000
mean        19.577083
std         12.398375
min          1.000000
25%          9.000000
50%         17.000000
75%         29.000000
max         70.000000
Name: duration_months, dtype: float64
```

[13]:
```
loan_level_path = PROCESSED_DIR / "loan_level_survival_base.csv"
df.to_csv(loan_level_path, index=False)
```

```
print(f"Saved loan-level base to: {loan_level_path}")
```

```
Saved loan-level base to: data/processed/loan_level_survival_base.csv
```

[14]:
```
static_cols = [
    "loan_amnt",
    "term_num",
    "installment",
    "emp_length",
    "home_ownership",
    "annual_inc",
    "verification_status",
    "purpose",
    "addr_state",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
]

def expand_loan(row):
    n = int(row["duration_months"])
    event = int(row["event"])

    out = pd.DataFrame({
        "id": row["id"],
        "issue_d": row["issue_d"],
        "month_on_book": np.arange(1, n + 1),
        "duration_months": row["duration_months"],
        "event": row["event"],
    })

    out["hazard_target"] = 0
    if event == 1:
        out.loc[out["month_on_book"] == n, "hazard_target"] = 1

    for col in static_cols:
        out[col] = row[col]
```

```
    return out
```

[15]:
```
panel_parts = []
for _, row in df.iterrows():
    panel_parts.append(expand_loan(row))

panel = pd.concat(panel_parts, ignore_index=True)

print("Panel shape:", panel.shape)
panel.head()
```

```
Panel shape: (978815, 27)
```

[15]:
```
         id    issue_d  month_on_book  duration_months  event  hazard_target  \
0  39651438 2015-02-01              1               49      0              0
1  39651438 2015-02-01              2               49      0              0
2  39651438 2015-02-01              3               49      0              0
3  39651438 2015-02-01              4               49      0              0
4  39651438 2015-02-01              5               49      0              0

   loan_amnt  term_num  installment emp_length home_ownership  annual_inc  \
0    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
1    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
2    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
3    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0
4    32000.0      60.0       687.65  10+ years       MORTGAGE    120000.0

  verification_status             purpose addr_state    dti  delinq_2yrs  \
0            Verified  debt_consolidation         CA  24.05          0.0
1            Verified  debt_consolidation         CA  24.05          0.0
2            Verified  debt_consolidation         CA  24.05          0.0
3            Verified  debt_consolidation         CA  24.05          0.0
4            Verified  debt_consolidation         CA  24.05          0.0

   inq_last_6mths  open_acc  pub_rec  revol_bal  revol_util  total_acc  \
0             0.0      20.0      0.0    39687.0        57.8       42.0
1             0.0      20.0      0.0    39687.0        57.8       42.0
2             0.0      20.0      0.0    39687.0        57.8       42.0
3             0.0      20.0      0.0    39687.0        57.8       42.0
4             0.0      20.0      0.0    39687.0        57.8       42.0

   mort_acc  pub_rec_bankruptcies  fico_avg  credit_history_years
0       2.0                   0.0     737.0             33.336071
1       2.0                   0.0     737.0             33.336071
2       2.0                   0.0     737.0             33.336071
3       2.0                   0.0     737.0             33.336071
4       2.0                   0.0     737.0             33.336071
```

[16]:
```
panel["mob_6bin"] = pd.cut(
    panel["month_on_book"],
    bins=[0, 6, 12, 24, 36, 60, 120],
    labels=["1_6", "7_12", "13_24", "25_36", "37_60", "61_plus"]
)

panel["month_on_book_sq"] = panel["month_on_book"] ** 2

panel[["id", "month_on_book", "mob_6bin", "hazard_target"]].head(20)
```

[16]:

	id	month_on_book	mob_6bin	hazard_target
0	39651438	1	1_6	0
1	39651438	2	1_6	0
2	39651438	3	1_6	0
3	39651438	4	1_6	0
4	39651438	5	1_6	0
5	39651438	6	1_6	0
6	39651438	7	7_12	0
7	39651438	8	7_12	0
8	39651438	9	7_12	0
9	39651438	10	7_12	0
10	39651438	11	7_12	0
11	39651438	12	7_12	0
12	39651438	13	13_24	0
13	39651438	14	13_24	0
14	39651438	15	13_24	0
15	39651438	16	13_24	0
16	39651438	17	13_24	0
17	39651438	18	13_24	0
18	39651438	19	13_24	0
19	39651438	20	13_24	0

[ ]:
```
# More flexible time features + interactions

panel["mob_str"] = panel["month_on_book"].astype(str)

panel["fico_x_mob"] = panel["fico_avg"] * panel["month_on_book"]
panel["term_x_mob"] = panel["term_num"] * panel["month_on_book"]
panel["dti_x_mob"] = panel["dti"] * panel["month_on_book"]

panel[[
    "id", "month_on_book", "mob_str",
    "fico_avg", "fico_x_mob",
    "term_num", "term_x_mob",
    "dti", "dti_x_mob"
]].head()
```

[17]:
```
panel_path = PROCESSED_DIR / "loan_month_panel.csv"
panel.to_csv(panel_path, index=False)
print(f"Saved panel to: {panel_path}")
```

```
Saved panel to: data/processed/loan_month_panel.csv
```

[18]:
```
train_panel = panel[panel["issue_d"] < "2016-01-01"].copy()
valid_panel = panel[(panel["issue_d"] >= "2016-01-01") & (panel["issue_d"] <␣
↪"2017-01-01")].copy()
test_panel  = panel[panel["issue_d"] >= "2017-01-01"].copy()

print("Train shape:", train_panel.shape)
print("Valid shape:", valid_panel.shape)
print("Test shape :", test_panel.shape)
```

```
Train shape: (517939, 29)
Valid shape: (218609, 29)
Test shape : (242267, 29)
```

[19]:
```
target = "hazard_target"

numeric_features = [
    "month_on_book",
    "loan_amnt",
    "term_num",
    "installment",
    "annual_inc",
    "dti",
    "delinq_2yrs",
    "inq_last_6mths",
    "open_acc",
    "pub_rec",
    "revol_bal",
    "revol_util",
    "total_acc",
    "mort_acc",
    "pub_rec_bankruptcies",
    "fico_avg",
    "credit_history_years",
]

categorical_features = [
    "emp_length",
    "home_ownership",
    "verification_status",
    "purpose",
    "addr_state",
    "mob_6bin",
```

```
]
```

[20]:
```
X_train = train_panel[numeric_features + categorical_features]
y_train = train_panel[target]

X_valid = valid_panel[numeric_features + categorical_features]
y_valid = valid_panel[target]

X_test = test_panel[numeric_features + categorical_features]
y_test = test_panel[target]

print(X_train.shape, y_train.shape)
print(X_valid.shape, y_valid.shape)
print(X_test.shape, y_test.shape)
```

```
(517939, 23) (517939,)
(218609, 23) (218609,)
(242267, 23) (242267,)
```

[21]:
```
numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features),
])
```

[22]:
```
hazard_model = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression(max_iter=1000))
])

hazard_model.fit(X_train, y_train)
```

[22]:
```
Pipeline(steps=[('prep',
                 ColumnTransformer(transformers=[('num',
                                                  Pipeline(steps=[('imputer',
SimpleImputer(strategy='median')),
                                                                  ('scaler',
StandardScaler())]),
                                                  ['month_on_book', 'loan_amnt',
```

```
                                                        'term_num', 'installment',
                                                        'annual_inc', 'dti',
                                                        'delinq_2yrs',
                                                        'inq_last_6mths', 'open_acc',
                                                        'pub_rec', 'revol_bal',
                                                        'revol_util', 'total_acc',
                                                        'mort_acc',
                                                        'pub_rec_bankruptcies',
                                                        'fico_avg',
                                                        'credit_history_years']),
                                                       ('cat',
                                                        Pipeline(steps=[('imputer',
SimpleImputer(strategy='most_frequent')),
                                                                        ('onehot',
OneHotEncoder(handle_unknown='ignore'))]),
                                                        ['emp_length',
                                                        'home_ownership',
                                                        'verification_status',
                                                        'purpose', 'addr_state',
                                                        'mob_6bin'])])),
                 ('model', LogisticRegression(max_iter=1000))])
```

[23]:
```
valid_pred_hazard = hazard_model.predict_proba(X_valid)[:, 1]
test_pred_hazard = hazard_model.predict_proba(X_test)[:, 1]

hazard_results = {
    "valid_roc_auc": roc_auc_score(y_valid, valid_pred_hazard),
    "valid_pr_auc": average_precision_score(y_valid, valid_pred_hazard),
    "test_roc_auc": roc_auc_score(y_test, test_pred_hazard),
    "test_pr_auc": average_precision_score(y_test, test_pred_hazard),
}

hazard_results
```

[23]:
```
{'valid_roc_auc': 0.6611623344575871,
 'valid_pr_auc': 0.013466118298246742,
 'test_roc_auc': 0.6609768148992778,
 'test_pr_auc': 0.008463529277463536}
```

[24]:
```
from sklearn.calibration import CalibratedClassifierCV
```

[25]:
```
rf_base = Pipeline([
    ("prep", preprocessor),
    ("model", RandomForestClassifier(
        n_estimators=200,
        min_samples_leaf=50,
        random_state=RANDOM_STATE,
```

```
        n_jobs=-1,
        class_weight="balanced_subsample"
    )),
])

rf_calibrated = CalibratedClassifierCV(
    estimator=rf_base,
    method="sigmoid",
    cv=3
)

rf_calibrated.fit(X_train, y_train)
```

[25]:
```
CalibratedClassifierCV(cv=3,
                       estimator=Pipeline(steps=[('prep',
ColumnTransformer(transformers=[('num',
   Pipeline(steps=[('imputer',
                    SimpleImputer(strategy='median')),
                   ('scaler',
                    StandardScaler())]),
   ['month_on_book',
    'loan_amnt',
    'term_num',
    'installment',
    'annual_inc',
    'dti',
    'delinq_2yrs',
    'inq_last_6mths',
    'open_acc',
    'pub_rec',
    'revol_bal',
    'revol_util',
    'total_a…
  ('cat',
   Pipeline(steps=[('imputer',
                    SimpleImputer(strategy='most_frequent')),
                   ('onehot',
                    OneHotEncoder(handle_unknown='ignore'))]),
   ['emp_length',
    'home_ownership',
    'verification_status',
    'purpose',
    'addr_state',
    'mob_6bin'])])),
                                                    ('model',
RandomForestClassifier(class_weight='balanced_subsample',
min_samples_leaf=50,
```

```
n_estimators=200,
n_jobs=-1,
random_state=42))]))
```

[26]:
```
valid_pred_rf = rf_calibrated.predict_proba(X_valid)[:, 1]
test_pred_rf = rf_calibrated.predict_proba(X_test)[:, 1]

rf_results = {
    "valid_roc_auc": roc_auc_score(y_valid, valid_pred_rf),
    "valid_pr_auc": average_precision_score(y_valid, valid_pred_rf),
    "test_roc_auc": roc_auc_score(y_test, test_pred_rf),
    "test_pr_auc": average_precision_score(y_test, test_pred_rf),
}

rf_results
```

[26]:
```
{'valid_roc_auc': 0.6557979652295732,
 'valid_pr_auc': 0.013410932948459013,
 'test_roc_auc': 0.6695228184965927,
 'test_pr_auc': 0.00856726564928971}
```

[27]:
```
results_df = pd.DataFrame([
    {
        "model": "logistic_hazard",
        **hazard_results
    },
    {
        "model": "random_forest",
        **rf_results
    }
])

results_df
```

[27]:

	model	valid_roc_auc	valid_pr_auc	test_roc_auc	test_pr_auc
0	logistic_hazard	0.661162	0.013466	0.660977	0.008464
1	random_forest	0.655798	0.013411	0.669523	0.008567

[28]:
```
test_scored = test_panel.copy()
test_scored["pred_hazard_logit"] = test_pred_hazard
test_scored["pred_hazard_rf"] = test_pred_rf

test_scored = test_scored.sort_values(["id", "month_on_book"]).copy()
test_scored.head()
```

[28]:
```
                id    issue_d  month_on_book  duration_months  event  \
214997  100001365 2017-03-01              1               24      0
```

214998	100001365	2017-03-01	2	24	0
214999	100001365	2017-03-01	3	24	0
215000	100001365	2017-03-01	4	24	0
215001	100001365	2017-03-01	5	24	0

	hazard_target	loan_amnt	term_num	installment	emp_length	\
214997	0	20000.0	36.0	683.46	10+ years	
214998	0	20000.0	36.0	683.46	10+ years	
214999	0	20000.0	36.0	683.46	10+ years	
215000	0	20000.0	36.0	683.46	10+ years	
215001	0	20000.0	36.0	683.46	10+ years	

	home_ownership	annual_inc	verification_status	purpose	\
214997	MORTGAGE	87000.0	Not Verified	debt_consolidation	
214998	MORTGAGE	87000.0	Not Verified	debt_consolidation	
214999	MORTGAGE	87000.0	Not Verified	debt_consolidation	
215000	MORTGAGE	87000.0	Not Verified	debt_consolidation	
215001	MORTGAGE	87000.0	Not Verified	debt_consolidation	

	addr_state	dti	delinq_2yrs	inq_last_6mths	open_acc	pub_rec	\
214997	MT	25.3	0.0	0.0	12.0	0.0	
214998	MT	25.3	0.0	0.0	12.0	0.0	
214999	MT	25.3	0.0	0.0	12.0	0.0	
215000	MT	25.3	0.0	0.0	12.0	0.0	
215001	MT	25.3	0.0	0.0	12.0	0.0	

	revol_bal	revol_util	total_acc	mort_acc	pub_rec_bankruptcies	\
214997	19262.0	79.3	19.0	1.0	0.0	
214998	19262.0	79.3	19.0	1.0	0.0	
214999	19262.0	79.3	19.0	1.0	0.0	
215000	19262.0	79.3	19.0	1.0	0.0	
215001	19262.0	79.3	19.0	1.0	0.0	

	fico_avg	credit_history_years	mob_6bin	month_on_book_sq	\
214997	662.0	12.495551	1_6	1	
214998	662.0	12.495551	1_6	4	
214999	662.0	12.495551	1_6	9	
215000	662.0	12.495551	1_6	16	
215001	662.0	12.495551	1_6	25	

	pred_hazard_logit	pred_hazard_rf
214997	0.003661	0.002667
214998	0.003654	0.002742
214999	0.003646	0.003279
215000	0.003638	0.003386
215001	0.003630	0.003490

[29]:
```
test_scored["survival_prob_logit"] = test_scored.
 ↪groupby("id")["pred_hazard_logit"].transform(
    lambda x: (1 - x).cumprod()
)
test_scored["cum_pd_logit"] = 1 - test_scored["survival_prob_logit"]

test_scored["survival_prob_rf"] = test_scored.groupby("id")["pred_hazard_rf"].
 ↪transform(
    lambda x: (1 - x).cumprod()
)
test_scored["cum_pd_rf"] = 1 - test_scored["survival_prob_rf"]

test_scored[[
    "id", "month_on_book", "pred_hazard_logit", "cum_pd_logit",
    "pred_hazard_rf", "cum_pd_rf"
]].head(20)
```

[29]:
```
               id  month_on_book  pred_hazard_logit  cum_pd_logit  \
214997  100001365              1           0.003661      0.003661
214998  100001365              2           0.003654      0.007302
214999  100001365              3           0.003646      0.010921
215000  100001365              4           0.003638      0.014518
215001  100001365              5           0.003630      0.018095
215002  100001365              6           0.003622      0.021652
215003  100001365              7           0.008429      0.029898
215004  100001365              8           0.008410      0.038057
215005  100001365              9           0.008392      0.046129
215006  100001365             10           0.008374      0.054117
215007  100001365             11           0.008356      0.062020
215008  100001365             12           0.008338      0.069841
215009  100001365             13           0.009706      0.078869
215010  100001365             14           0.009685      0.087790
215011  100001365             15           0.009664      0.096606
215012  100001365             16           0.009643      0.105318
215013  100001365             17           0.009622      0.113927
215014  100001365             18           0.009602      0.122435
215015  100001365             19           0.009581      0.130842
215016  100001365             20           0.009560      0.139151

        pred_hazard_rf  cum_pd_rf
214997        0.002667   0.002667
214998        0.002742   0.005402
214999        0.003279   0.008663
215000        0.003386   0.012020
215001        0.003490   0.015468
215002        0.004169   0.019572
215003        0.007933   0.027350
```

```
215004        0.008054   0.035183
215005        0.008221   0.043115
215006        0.008753   0.051491
215007        0.008793   0.059831
215008        0.008816   0.068120
215009        0.010288   0.077707
215010        0.010497   0.087388
215011        0.011407   0.097799
215012        0.011838   0.108479
215013        0.012446   0.119576
215014        0.012149   0.130272
215015        0.011887   0.140611
215016        0.012213   0.151107
```

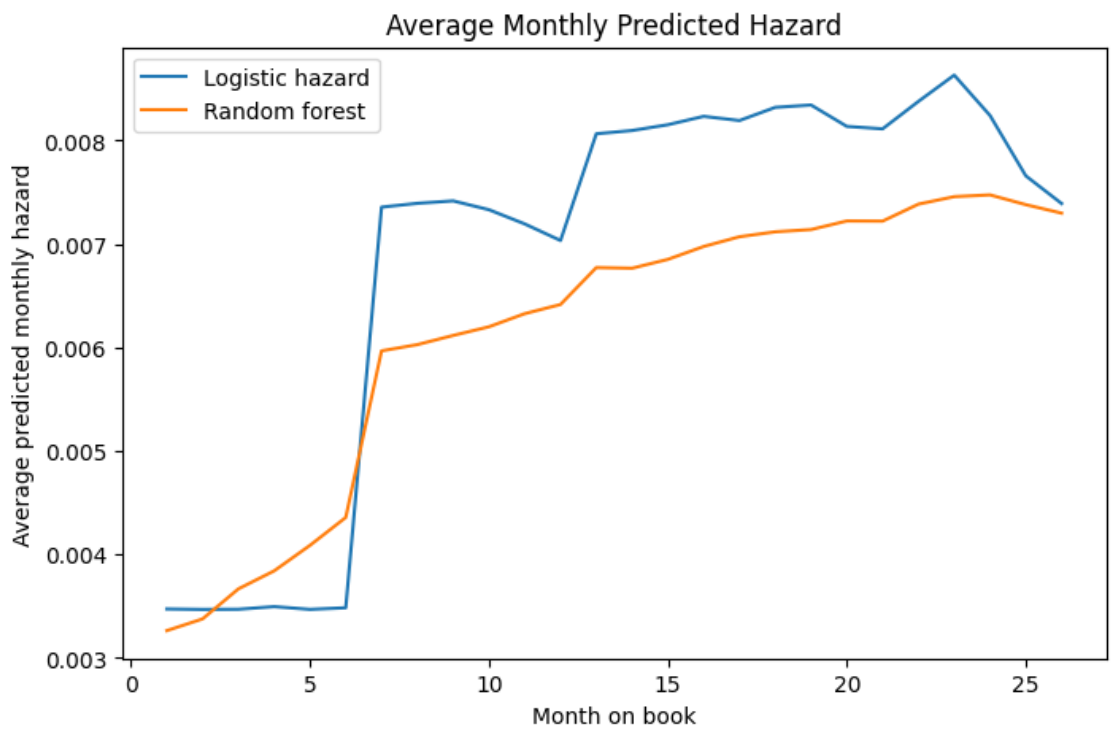
[30]:
```
pd_summary = (
    test_scored[test_scored["month_on_book"].isin([6, 12, 24, 36])]
    .groupby("month_on_book")
    .agg(
        avg_cum_pd_logit=("cum_pd_logit", "mean"),
        avg_cum_pd_rf=("cum_pd_rf", "mean"),
        loans=("id", "nunique"),
    )
    .reset_index()
)

pd_summary
```

[30]:

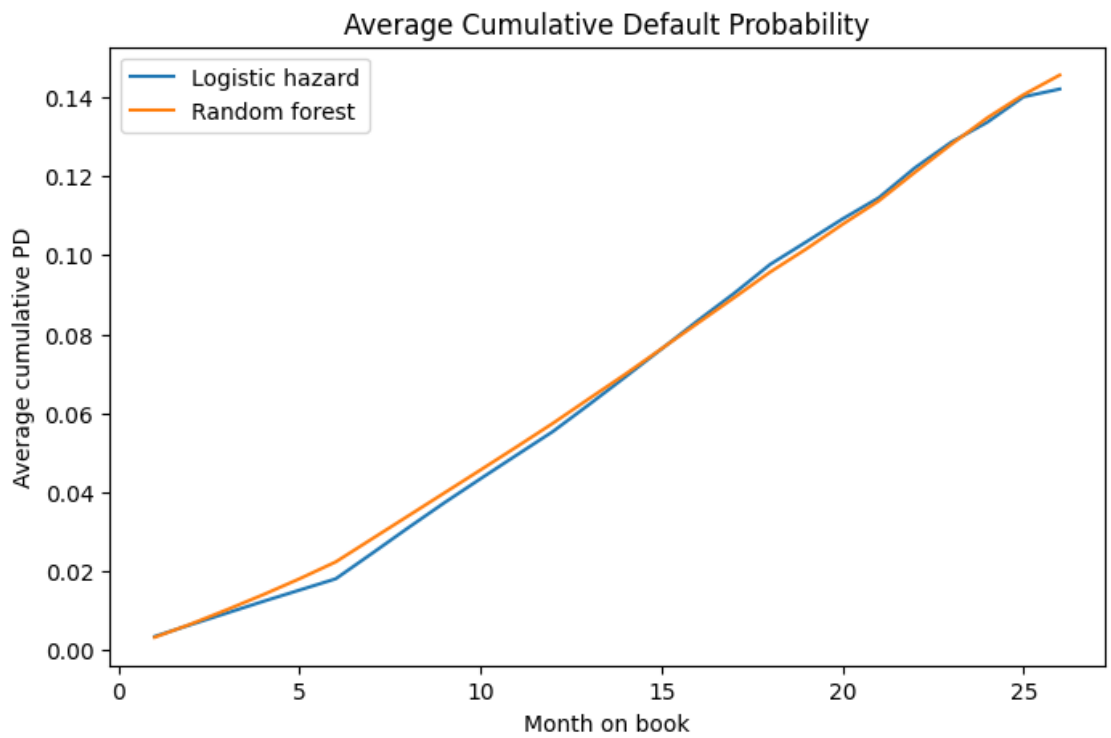
	month_on_book	avg_cum_pd_logit	avg_cum_pd_rf	loans
0	6	0.018066	0.022346	16402
1	12	0.055362	0.057439	9504
2	24	0.133646	0.134771	1103

[31]:
```
plot_hazard = (
    test_scored.groupby("month_on_book")[["pred_hazard_logit",␣
 ↪"pred_hazard_rf"]]
    .mean()
    .reset_index()
)

plt.figure(figsize=(8, 5))
plt.plot(plot_hazard["month_on_book"], plot_hazard["pred_hazard_logit"],␣
 ↪label="Logistic hazard")
plt.plot(plot_hazard["month_on_book"], plot_hazard["pred_hazard_rf"],␣
 ↪label="Random forest")
plt.xlabel("Month on book")
plt.ylabel("Average predicted monthly hazard")
plt.title("Average Monthly Predicted Hazard")
```

```
plt.legend()
plt.show()
```



[32]:
```
plot_cum_pd = (
    test_scored.groupby("month_on_book")[["cum_pd_logit", "cum_pd_rf"]]
    .mean()
    .reset_index()
)

plt.figure(figsize=(8, 5))
plt.plot(plot_cum_pd["month_on_book"], plot_cum_pd["cum_pd_logit"],␣
 ↪label="Logistic hazard")
plt.plot(plot_cum_pd["month_on_book"], plot_cum_pd["cum_pd_rf"], label="Random␣
 ↪forest")
plt.xlabel("Month on book")
plt.ylabel("Average cumulative PD")
plt.title("Average Cumulative Default Probability")
plt.legend()
plt.show()
```



[33]:
```
loan_12m = test_scored[test_scored["month_on_book"] == 12].copy()

print("Loans with 12-month horizon rows:", loan_12m["id"].nunique())
loan_12m[["id", "cum_pd_logit", "cum_pd_rf"]].head()
```

```
Loans with 12-month horizon rows: 9504
```

[33]:

	id	cum_pd_logit	cum_pd_rf
215008	100001365	0.069841	0.068120
539498	100001532	0.022698	0.043225
62778	100002096	0.056001	0.058406
86881	100003912	0.042560	0.048000
866480	100005538	0.064972	0.072479

[34]:
```
LGD = 0.60

loan_12m["ead_proxy"] = loan_12m["loan_amnt"]
loan_12m["expected_loss_12m_logit"] = loan_12m["cum_pd_logit"] * LGD *␣
↪loan_12m["ead_proxy"]
loan_12m["expected_loss_12m_rf"] = loan_12m["cum_pd_rf"] * LGD *␣
↪loan_12m["ead_proxy"]

loan_12m[[
```

```
    "id",
    "loan_amnt",
    "cum_pd_logit",
    "cum_pd_rf",
    "expected_loss_12m_logit",
    "expected_loss_12m_rf"
]].head()
```

[34]:

```
                id  loan_amnt  cum_pd_logit  cum_pd_rf  \
215008  100001365    20000.0      0.069841   0.068120
539498  100001532    30000.0      0.022698   0.043225
62778   100002096    20000.0      0.056001   0.058406
86881   100003912    28500.0      0.042560   0.048000
866480  100005538    14400.0      0.064972   0.072479

        expected_loss_12m_logit  expected_loss_12m_rf
215008               838.089537            817.438930
539498               408.571948            778.049887
62778                672.014933            700.871312
86881                727.773786            820.807290
866480               561.358482            626.215936
```

[35]:

```
funding_cost = 0.04
profit_margin = 0.03

loan_12m["el_rate_12m_logit"] = loan_12m["expected_loss_12m_logit"] /␣
 ↪loan_12m["loan_amnt"]
loan_12m["el_rate_12m_rf"] = loan_12m["expected_loss_12m_rf"] /␣
 ↪loan_12m["loan_amnt"]

loan_12m["required_rate_logit"] = funding_cost + profit_margin +␣
 ↪loan_12m["el_rate_12m_logit"]
loan_12m["required_rate_rf"] = funding_cost + profit_margin +␣
 ↪loan_12m["el_rate_12m_rf"]

loan_12m[[
    "id",
    "required_rate_logit",
    "required_rate_rf"
]].head()
```

[35]:

```
                id  required_rate_logit  required_rate_rf
215008  100001365             0.111904          0.110872
539498  100001532             0.083619          0.095935
62778   100002096             0.103601          0.105044
86881   100003912             0.095536          0.098800
866480  100005538             0.108983          0.113487
```

[36]:
```
loan_12m["risk_band_logit"] = pd.qcut(
    loan_12m["cum_pd_logit"],
    q=5,
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

reserve_summary_logit = (
    loan_12m.groupby("risk_band_logit")
    .agg(
        loans=("id", "count"),
        avg_pd_12m=("cum_pd_logit", "mean"),
        total_expected_loss=("expected_loss_12m_logit", "sum"),
        avg_required_rate=("required_rate_logit", "mean"),
    )
    .reset_index()
)

reserve_summary_logit
```

[36]:

	risk_band_logit	loans	avg_pd_12m	total_expected_loss	avg_required_rate
0	Very Low	1901	0.022054	3.577197e+05	0.083233
1	Low	1901	0.036974	5.877627e+05	0.092184
2	Medium	1900	0.049112	8.074657e+05	0.099467
3	High	1901	0.064115	1.093634e+06	0.108469
4	Very High	1901	0.104550	2.176245e+06	0.132730

[37]:
```
loan_12m["risk_band_rf"] = pd.qcut(
    loan_12m["cum_pd_rf"],
    q=5,
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

reserve_summary_rf = (
    loan_12m.groupby("risk_band_rf")
    .agg(
        loans=("id", "count"),
        avg_pd_12m=("cum_pd_rf", "mean"),
        total_expected_loss=("expected_loss_12m_rf", "sum"),
        avg_required_rate=("required_rate_rf", "mean"),
    )
    .reset_index()
)

reserve_summary_rf
```

[37]:

	risk_band_rf	loans	avg_pd_12m	total_expected_loss	avg_required_rate
0	Very Low	1901	0.036193	5.813638e+05	0.091716

```
1          Low   1901    0.045344         7.230147e+05           0.097206
2       Medium   1900    0.053556         8.747815e+05           0.102133
3         High   1901    0.064262         1.123034e+06           0.108557
4    Very High   1901    0.087837         1.794889e+06           0.122702
```

[38]:
```
test_scored_path = PROCESSED_DIR / "test_panel_scored.csv"
loan_12m_path = PROCESSED_DIR / "loan_level_12m_outputs.csv"
results_path = PROCESSED_DIR / "model_comparison_results.csv"

test_scored.to_csv(test_scored_path, index=False)
loan_12m.to_csv(loan_12m_path, index=False)
results_df.to_csv(results_path, index=False)

print(f"Saved scored panel to: {test_scored_path}")
print(f"Saved 12m loan outputs to: {loan_12m_path}")
print(f"Saved model comparison to: {results_path}")
```

```
Saved scored panel to: data/processed/test_panel_scored.csv
Saved 12m loan outputs to: data/processed/loan_level_12m_outputs.csv
Saved model comparison to: data/processed/model_comparison_results.csv
```

[39]:
```
print("Model comparison:")
display(results_df)

print("\n12-month cumulative PD summary:")
display(pd_summary)

print("\nReserve summary (logistic hazard):")
display(reserve_summary_logit)
```

```
Model comparison:

             model  valid_roc_auc  valid_pr_auc  test_roc_auc  test_pr_auc
0  logistic_hazard       0.661162      0.013466      0.660977     0.008464
1    random_forest       0.655798      0.013411      0.669523     0.008567


12-month cumulative PD summary:

   month_on_book  avg_cum_pd_logit  avg_cum_pd_rf  loans
0              6          0.018066       0.022346  16402
1             12          0.055362       0.057439   9504
2             24          0.133646       0.134771   1103


Reserve summary (logistic hazard):

  risk_band_logit  loans  avg_pd_12m  total_expected_loss  avg_required_rate
0        Very Low   1901    0.022054         3.577197e+05           0.083233
1             Low   1901    0.036974         5.877627e+05           0.092184
2          Medium   1900    0.049112         8.074657e+05           0.099467
```

```
3            High   1901    0.064115         1.093634e+06           0.108469
4       Very High   1901    0.104550         2.176245e+06           0.132730
```

[40]:
```python
loan_12m["risk_band_rf"] = pd.qcut(
    loan_12m["cum_pd_rf"],
    q=5,
    labels=["Very Low", "Low", "Medium", "High", "Very High"]
)

reserve_summary_rf = (
    loan_12m.groupby("risk_band_rf")
    .agg(
        loans=("id", "count"),
        avg_pd_12m=("cum_pd_rf", "mean"),
        total_expected_loss=("expected_loss_12m_rf", "sum"),
        avg_required_rate=("required_rate_rf", "mean"),
    )
    .reset_index()
)

reserve_summary_rf
```

[40]:

	risk_band_rf	loans	avg_pd_12m	total_expected_loss	avg_required_rate
0	Very Low	1901	0.036193	5.813638e+05	0.091716
1	Low	1901	0.045344	7.230147e+05	0.097206
2	Medium	1900	0.053556	8.747815e+05	0.102133
3	High	1901	0.064262	1.123034e+06	0.108557
4	Very High	1901	0.087837	1.794889e+06	0.122702

14 Looking at test

[41]:
```python
test_loans = df[df["issue_d"] >= "2017-01-01"].copy()

loan_obs = test_loans[["id", "duration_months", "event"]].drop_duplicates().
 ↪copy()

print(loan_obs.shape)
loan_obs.head()
```

```
(20781, 3)
```

[41]:

	id	duration_months	event
2066630	125356772	10	0
477199	128490686	13	0
641990	109655666	14	0
507598	127049010	14	0
2129008	121746633	17	0

[42]:
```
max_mob = int(test_scored["month_on_book"].max())
months = np.arange(1, max_mob + 1)

n_loans = loan_obs["id"].nunique()

observed_curve = pd.DataFrame({"month_on_book": months})

observed_curve["observed_cum_pd"] = [
    ((loan_obs["event"] == 1) & (loan_obs["duration_months"] <= m)).sum() /␣
 ↪n_loans
    for m in months
]

observed_curve.head(15)
```

[42]:

	month_on_book	observed_cum_pd
0	1	0.001732
1	2	0.004283
2	3	0.007266
3	4	0.011693
4	5	0.015495
5	6	0.019200
6	7	0.023242
7	8	0.026659
8	9	0.030124
9	10	0.033396
10	11	0.036524
11	12	0.040133
12	13	0.042250
13	14	0.044608
14	15	0.046340

[43]:
```
plot_cum_pd = (
    test_scored.groupby("month_on_book")[["cum_pd_logit", "cum_pd_rf"]]
    .mean()
    .reset_index()
)
```

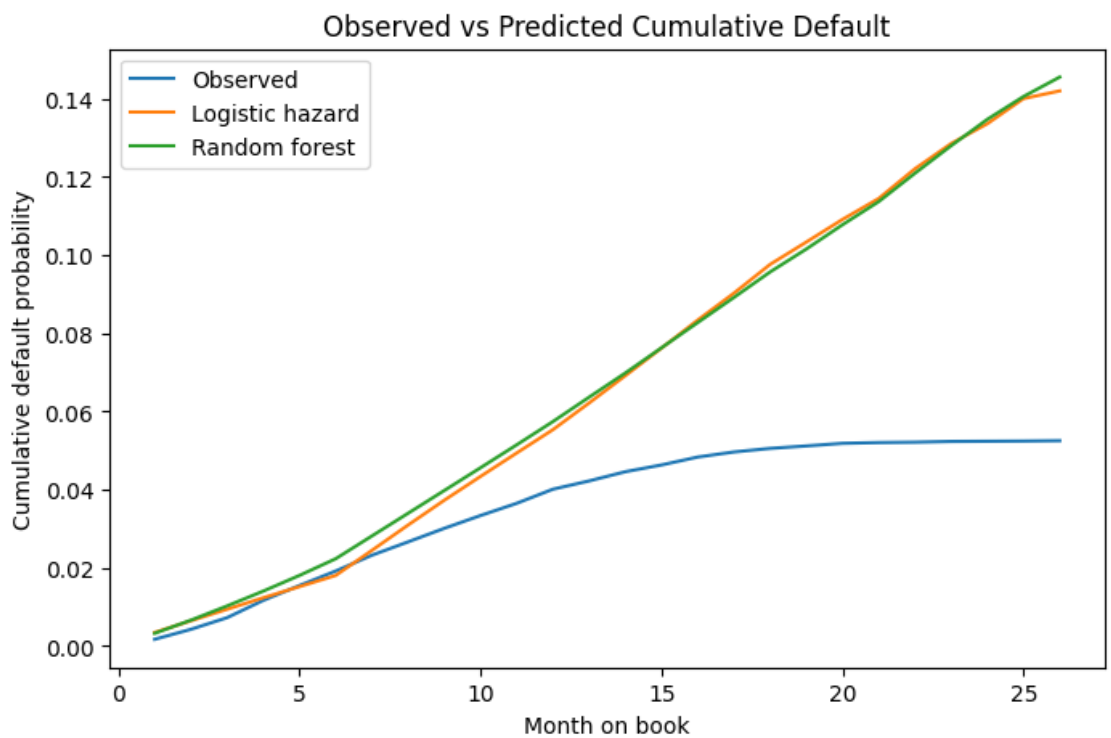
[44]:
```
compare_curve = observed_curve.merge(
    plot_cum_pd,
    on="month_on_book",
    how="inner"
)

compare_curve.head()
```

[44]:

	month_on_book	observed_cum_pd	cum_pd_logit	cum_pd_rf
0	1	0.001732	0.003474	0.003264
1	2	0.004283	0.006470	0.006627
2	3	0.007266	0.009418	0.010268
3	4	0.011693	0.012360	0.014065
4	5	0.015495	0.015211	0.018076

[45]:

```
plt.figure(figsize=(8, 5))
plt.plot(compare_curve["month_on_book"], compare_curve["observed_cum_pd"],␣
 ↪label="Observed")
plt.plot(compare_curve["month_on_book"], compare_curve["cum_pd_logit"],␣
 ↪label="Logistic hazard")
plt.plot(compare_curve["month_on_book"], compare_curve["cum_pd_rf"],␣
 ↪label="Random forest")
plt.xlabel("Month on book")
plt.ylabel("Cumulative default probability")
plt.title("Observed vs Predicted Cumulative Default")
plt.legend()
plt.show()
```



[46]:

```
milestones = [6, 12, 18, 24]

calibration_table = compare_curve[
```

```
    compare_curve["month_on_book"].isin(milestones)
].copy()

calibration_table["logit_error"] = (
    calibration_table["cum_pd_logit"] - calibration_table["observed_cum_pd"]
)

calibration_table["rf_error"] = (
    calibration_table["cum_pd_rf"] - calibration_table["observed_cum_pd"]
)

calibration_table
```

[46]:

```
    month_on_book  observed_cum_pd  cum_pd_logit  cum_pd_rf  logit_error  \
5               6         0.019200      0.018066   0.022346    -0.001134
11             12         0.040133      0.055362   0.057439     0.015229
17             18         0.050575      0.097656   0.095718     0.047081
23             24         0.052404      0.133646   0.134771     0.081242

    rf_error
5   0.003145
11  0.017306
17  0.045143
23  0.082367
```

[47]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from lifelines import KaplanMeierFitter

# ==========================================================
# 1. Build loan-level TEST set for Kaplan-Meier
# ==========================================================

# Make sure issue_d is datetime
df["issue_d"] = pd.to_datetime(df["issue_d"], errors="coerce")

# Match your test split
test_loans = df[df["issue_d"] >= "2017-01-01"].copy()

# Keep one row per loan with survival inputs
loan_obs = (
    test_loans[["id", "duration_months", "event"]]
    .drop_duplicates(subset=["id"])
    .copy()
)
```

```
# Clean types
loan_obs["duration_months"] = pd.to_numeric(loan_obs["duration_months"],␣
↪errors="coerce")
loan_obs["event"] = pd.to_numeric(loan_obs["event"], errors="coerce")

# Drop bad rows if any
loan_obs = loan_obs.dropna(subset=["id", "duration_months", "event"]).copy()

# Optional: force valid ranges/types
loan_obs["duration_months"] = loan_obs["duration_months"].astype(int)
loan_obs["event"] = loan_obs["event"].astype(int)

print("Loan-level test sample shape:", loan_obs.shape)
print(loan_obs.head())


# ==========================================================
# 2. Fit Kaplan-Meier on observed test-loan outcomes
# ==========================================================

kmf = KaplanMeierFitter()

kmf.fit(
    durations=loan_obs["duration_months"],
    event_observed=loan_obs["event"],
    label="Kaplan-Meier survival"
)

# Convert survival curve to cumulative default curve
km_curve = kmf.survival_function_.reset_index()
km_curve.columns = ["month_on_book", "survival_prob"]
km_curve["observed_cum_pd_km"] = 1 - km_curve["survival_prob"]

print("\nKaplan-Meier curve:")
print(km_curve.head(15))


# ==========================================================
# 3. Build average model-predicted cumulative PD curve
# ==========================================================

plot_cum_pd = (
    test_scored.groupby("month_on_book")[["cum_pd_logit", "cum_pd_rf"]]
    .mean()
    .reset_index()
    .sort_values("month_on_book")
)
```

```python
print("\nPredicted cumulative PD curve:")
print(plot_cum_pd.head(15))


# ==========================================================
# 4. Merge KM observed curve with model curves
# ==========================================================

compare_curve = pd.merge(
    plot_cum_pd,
    km_curve[["month_on_book", "observed_cum_pd_km"]],
    on="month_on_book",
    how="left"
).sort_values("month_on_book")

print("\nMerged comparison curve:")
print(compare_curve.head(15))


# ==========================================================
# 5. Plot observed KM cumulative default vs predicted curves
# ==========================================================

plt.figure(figsize=(10, 6))
plt.plot(
    compare_curve["month_on_book"],
    compare_curve["observed_cum_pd_km"],
    label="Observed cumulative PD (Kaplan-Meier)"
)
plt.plot(
    compare_curve["month_on_book"],
    compare_curve["cum_pd_logit"],
    label="Predicted cumulative PD (Logistic hazard)"
)
plt.plot(
    compare_curve["month_on_book"],
    compare_curve["cum_pd_rf"],
    label="Predicted cumulative PD (Random forest)"
)

plt.xlabel("Month on book")
plt.ylabel("Cumulative default probability")
plt.title("Observed vs Predicted Cumulative Default Probability")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```python
# ==========================================================
# 6. Optional: milestone calibration table
# ==========================================================

milestones = [6, 12, 18, 24]

calibration_table = (
    compare_curve[compare_curve["month_on_book"].isin(milestones)]
    .copy()
)

calibration_table["logit_error"] = (
    calibration_table["cum_pd_logit"] - calibration_table["observed_cum_pd_km"]
)

calibration_table["rf_error"] = (
    calibration_table["cum_pd_rf"] - calibration_table["observed_cum_pd_km"]
)

print("\nCalibration table:")
print(calibration_table[[
    "month_on_book",
    "observed_cum_pd_km",
    "cum_pd_logit",
    "cum_pd_rf",
    "logit_error",
    "rf_error"
]])
```

```
Loan-level test sample shape: (20781, 3)
                id  duration_months  event
2066630  125356772               10      0
477199   128490686               13      0
641990   109655666               14      0
507598   127049010               14      0
2129008  121746633               17      0

Kaplan-Meier curve:
    month_on_book  survival_prob  observed_cum_pd_km
0             0.0       1.000000            0.000000
1             1.0       0.998268            0.001732
2             2.0       0.995668            0.004332
3             3.0       0.992590            0.007410
4             4.0       0.987773            0.012227
5             5.0       0.983358            0.016642
6             6.0       0.978742            0.021258
```

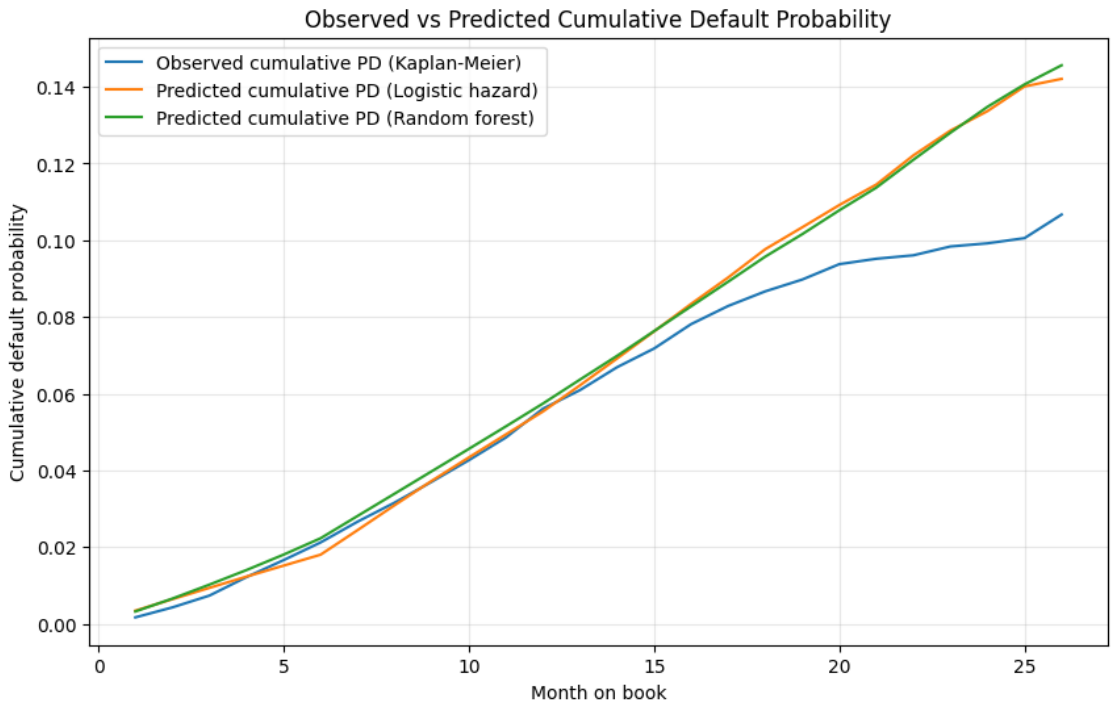
7	7.0	0.973322	0.026678
8	8.0	0.968358	0.031642
9	9.0	0.962924	0.037076
10	10.0	0.957323	0.042677
11	11.0	0.951409	0.048591
12	12.0	0.943901	0.056099
13	13.0	0.939060	0.060940
14	14.0	0.933096	0.066904

Predicted cumulative PD curve:

	month_on_book	cum_pd_logit	cum_pd_rf
0	1	0.003474	0.003264
1	2	0.006470	0.006627
2	3	0.009418	0.010268
3	4	0.012360	0.014065
4	5	0.015211	0.018076
5	6	0.018066	0.022346
6	7	0.024481	0.028165
7	8	0.030969	0.033978
8	9	0.037313	0.039801
9	10	0.043398	0.045623
10	11	0.049399	0.051490
11	12	0.055362	0.057439
12	13	0.062190	0.063681
13	14	0.069169	0.069867
14	15	0.076239	0.076319

Merged comparison curve:

	month_on_book	cum_pd_logit	cum_pd_rf	observed_cum_pd_km
0	1	0.003474	0.003264	0.001732
1	2	0.006470	0.006627	0.004332
2	3	0.009418	0.010268	0.007410
3	4	0.012360	0.014065	0.012227
4	5	0.015211	0.018076	0.016642
5	6	0.018066	0.022346	0.021258
6	7	0.024481	0.028165	0.026678
7	8	0.030969	0.033978	0.031642
8	9	0.037313	0.039801	0.037076
9	10	0.043398	0.045623	0.042677
10	11	0.049399	0.051490	0.048591
11	12	0.055362	0.057439	0.056099
12	13	0.062190	0.063681	0.060940
13	14	0.069169	0.069867	0.066904
14	15	0.076239	0.076319	0.071779




```
Calibration table:
    month_on_book  observed_cum_pd_km  cum_pd_logit  cum_pd_rf  logit_error  \
5               6            0.021258      0.018066   0.022346    -0.003192
11             12            0.056099      0.055362   0.057439    -0.000737
17             18            0.086677      0.097656   0.095718     0.010979
23             24            0.099196      0.133646   0.134771     0.034450

    rf_error
5   0.001087
11  0.001340
17  0.009041
23  0.035574
```